



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Aprendizaje Automático Probabilístico

Algebra lineal (actividades)

Alfons Juan

DSIC

Departamento de Sistemas
Informáticos y Computación

Índice

7.1	Introducción	1
7.1.1	Notación	2
7.1.2	Espacios vectoriales	3
7.1.3	Normas vectoriales y matriciales	4
7.1.3.1	Normas vectoriales	4
7.1.3.2	Normas matriciales	5
7.1.4	Propiedades de una matriz	6
7.1.5	Tipos de matrices especiales	7
7.2	Multiplicación de matrices	9
7.2.1	Productos vector-vector	10
7.2.2	Productos matriz-vector	11
7.2.3	Productos matriz-matriz	12
7.2.4	Aplicación: manipulación de matrices de datos	13
7.2.4.1	Suma de trozos de la matriz	15
7.2.4.2	Escalado de filas y columnas de una matriz	16
7.2.4.3	Suma de cuadrados y matriz de dispersión	17
7.2.4.4	Matriz de Gram	19
7.2.4.5	Matriz de distancias	20

7.3 Inversión de matrices	21
7.3.1 Inversa de una matriz cuadrada	21
7.4 Descomposición en valores propios (EVD)	22
7.4.1 Conceptos básicos	22
7.4.3 Valores y vectores propios de matriz simétrica	24
7.4.5 Estandarización y blanqueado de datos	25
7.5 Descomposición en valores singulares (SVD)	31
7.5.1 Conceptos básicos	31
7.5.2 Conexión entre la SVD y la EVD	32
7.5.5 SVD truncada	33
7.8 Cálculo matricial	34

7.1. Introducció

- ▶ Consulta la siguiente documentación sobre NumPy:
 - ▷ [NumPy quickstart](#)
 - ▷ [NumPy: the absolute basics for beginners](#)
 - ▷ [Array creation](#)
 - ▷ [Indexing on ndarrays](#)
 - ▷ [Data types](#)
 - ▷ [Broadcasting](#)
 - ▷ [Universal functions \(ufunc\) basics](#)
 - ▷ [Copies and views](#)

7.1.1. Notación

- Completa el script para crear un vector, matriz y tensor en numpy con los enteros $\{0, \dots, 7\}$

7.1.1.notation.py

```
1 import numpy as np
2 a = np.arange(8)
3 print(a, a.shape)
4 a2d = a.reshape([2, -1])
5 print(a2d, a2d.shape)
6 a3d = a.reshape([2, 2, -1])
7 print(a3d, a3d.shape)
```

7.1.1.notation.out

```
1 [0 1 2 3 4 5 6 7] (8,)
2 [[0 1 2 3]
3  [4 5 6 7]] (2, 4)
4 [[[0 1]
5  [2 3]]
6
7  [[4 5]
8  [6 7]]] (2, 2, 2)
```

7.1.2. Espacios vectoriales

- Completa el script para hallar el rango de diversas matrices mediante una función numpy apropiada

7.1.2.rank.py

```
1 import numpy as np
2 A = np.eye(4)
3 print(np.linalg.matrix_rank(A))
4 A[-1, -1] = 0.
5 print(np.linalg.matrix_rank(A))
6 A = np.ones((4,))
7 print(np.linalg.matrix_rank(A))
8 A = np.zeros((4,))
9 print(np.linalg.matrix_rank(A))
```

7.1.2.rank.out

```
1 4
2 3
3 1
4 0
```

7.1.3. Normas vectoriales y matriciales

7.1.3.1. Normas vectoriales

- Halla las normas L1, L2 y L ∞ de $\mathbf{x} = (-3, 4)^t$

$$\|\mathbf{x}\|_1 = |-3| + |4| = 7$$

$$\|\mathbf{x}\|_2 = \sqrt{(-3)^2 + 4^2} = 5$$

$$\|\mathbf{x}\|_\infty = \max |-3|, |4| = 4$$

- Completa el script para hacerlo con numpy

```
7.1.3.1.vecnorm.py  
1 import numpy as np  
2 a = np.array([-3, 4])  
3 print(np.linalg.norm(a, 1))  
4 print(np.linalg.norm(a, 2))  
5 print(np.linalg.norm(a, np.inf))
```

```
7.1.3.1.vecnorm.out  
1 7.0  
2 5.0  
3 4.0
```

7.1.3.2. Normas matriciales

- Halla la norma de Frobenius de $\mathbf{A} = \begin{pmatrix} -2 & -2 \\ 2 & 2 \end{pmatrix}$

$$\|\mathbf{A}\|_F = \|\text{vec}(\mathbf{A})\|_2 = \sqrt{(-2)^2 + (-2)^2 + 2^2 + 2^2} = 4$$

- Completa el script para hacerlo con numpy

```
7.1.3.2.matnorm.py  
1 import numpy as np  
2 A = np.array([[ -2, -2], [ 2,  2]])  
3 print(np.linalg.norm(A, 'fro'))
```

```
7.1.3.2.matnorm.out  
1 4.0
```


7.1.4. Propiedades de una matriz

- Completa el script para hallar la traza, determinante, rango y número de condición de una matriz dada

7.1.4.matprops.py

```
1 import numpy as np
2 A = np.array([[2, 1], [1, 2]])
3 print(np.trace(A))
4 print(round(np.linalg.det(A)))
5 print(np.linalg.matrix_rank(A))
6 print(round(np.linalg.cond(A)))
```

7.1.4.matprops.out

```
1 4
2 3
3 2
4 3
```

7.1.5. Tipos de matrices especiales

► Determina si $A = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix}$ es definida positiva o no

$\begin{pmatrix} a & b \\ b & d \end{pmatrix}$ es definida positiva sii $a > 0$, $d > 0$ y $ad > b^2$

$$a = \frac{5}{2} \rightarrow a > 0$$

$$d = \frac{5}{2} \rightarrow d > 0$$

$$ad = \frac{25}{4}, \quad b^2 = \frac{9}{4} \rightarrow ad > b^2$$

Sí es positiva

► Indica la afirmación incorrecta:

1. Dos vectores $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ son ortogonales si $\mathbf{x}^t \mathbf{y} \neq 0$
2. Un vector $\mathbf{x} \in \mathbb{R}^n$ está normalizado si $\|\mathbf{x}\|_2 = 1$
3. Un conjunto de vectores ortogonales dos a dos y normalizados se denomina ortonormal
4. Todas son correctas

La 1; es $\mathbf{x}^t \mathbf{y} = 0$

7.2. Multiplicación de matrices

- Considera las propiedades asociativa, distributiva y conmutativa.
La multiplicación de matrices:

1. Cumple las tres propiedades
2. Cumple dos de las tres
3. Cumple una de las tres
4. No cumple ninguna de las tres

La 2. Cumple la asociativa y distributiva, pero no la conmutativa.

7.2.1. Productos vector-vector

► Indica la afirmación incorrecta:

1. Dados $x, y \in \mathbb{R}^n$, su producto escalar es $x^t y = \sum_{i=1}^n x_i y_i$
2. Dados $x \in \mathbb{R}^m$ e $y \in \mathbb{R}^n$, su producto vectorial, xy^t , es una matriz $\mathbb{R}^{m \times n}$ con entrada ij igual a $(xy^t)_{ij} = x_i y_j$
3. Dados $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$ y $w \in \mathbb{R}^n$, entonces $(xy^t)w = (y^t w)x$
4. Todas son correctas

La 4. La 3 es fácil de probar mediante la asociativa:

$$\underbrace{(xy^t)}_{m \times n} \underbrace{w}_{n \times 1} = \underbrace{x}_{m \times 1} \underbrace{(y^t w)}_{1 \times 1} = (y^t w)x$$

7.2.2. Productos matriz-vector

► Dada una matriz $A \in \mathbb{R}^{m \times n}$ y un vector $x \in \mathbb{R}^n$, su producto, $y = Ax \in \mathbb{R}^m$, puede verse como una combinación lineal de:

1. De filas de A
2. De columnas de A
3. Tanto de filas como de columnas de A
4. Ninguna de las anteriores

La 2.

7.2.3. Productos matriz-matriz

- Dadas dos matrices, $\mathbf{A} \in \mathbb{R}^{m \times n}$ y $\mathbf{B} \in \mathbb{R}^{n \times p}$, existen cuatro maneras diferentes de interpretar su producto

$$\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$$

La interpretación usual del mismo es en términos de:

1. Filas por columnas
2. Columnas por filas
3. Columnas de $\mathbf{C}$ como productos matriz-vector de la matriz $\mathbf{A}$ y las columnas de $\mathbf{B}$
4. Filas de $\mathbf{C}$ como productos matriz-vector de las filas de $\mathbf{A}$ y la matriz $\mathbf{B}$

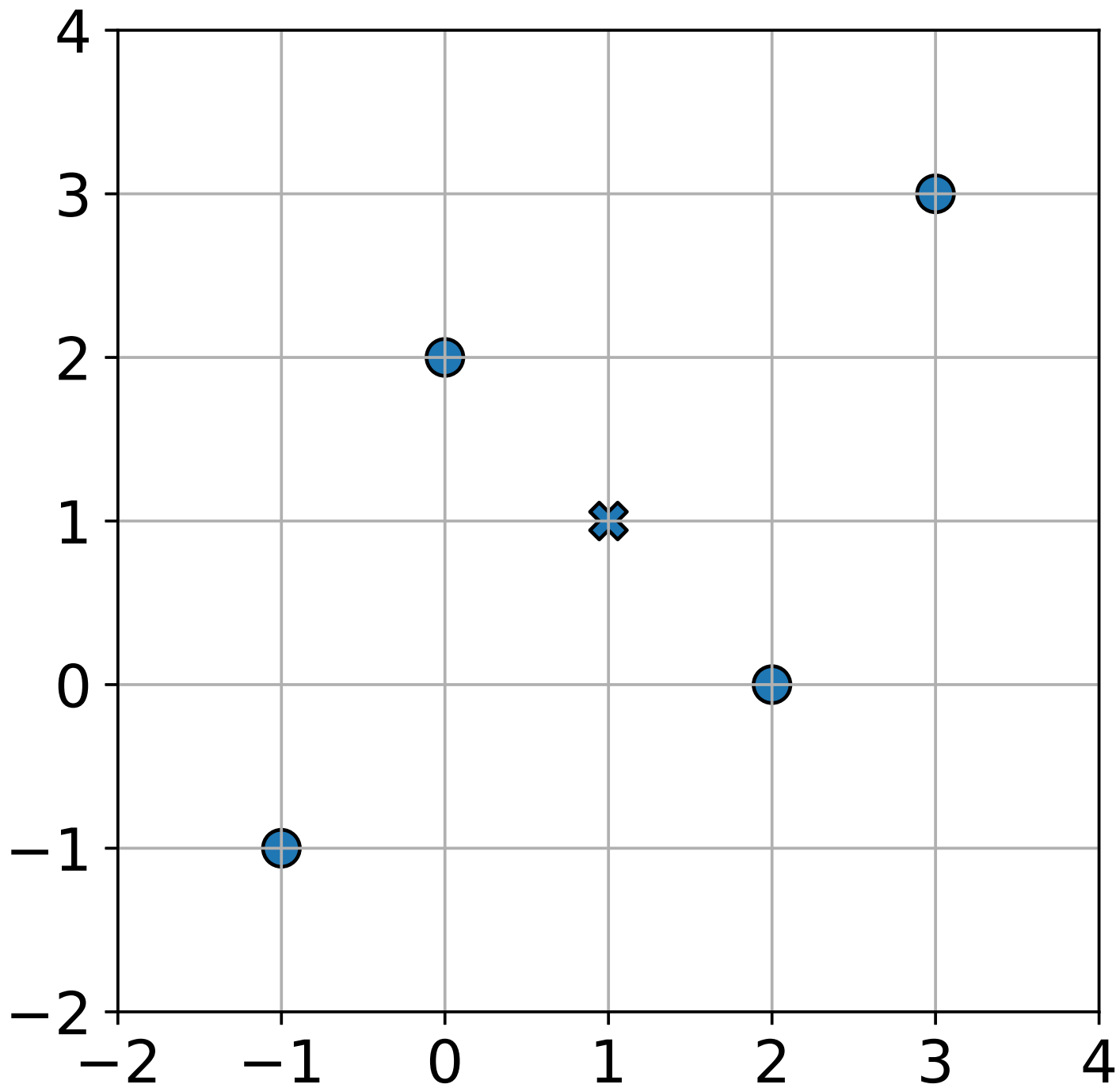
La 1.

7.2.4. Aplicación: manipulación de matrices de datos

- Completa el script para representar $X^t = \begin{bmatrix} -1 & 0 & 2 & 3 \\ -1 & 2 & 0 & 3 \end{bmatrix}$, $\bar{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

7.2.4.plotX.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
4 m = np.mean(X, axis=0)
5 fig, ax = plt.subplots()
6 ax.set_aspect("equal")
7 plt.grid(True)
8 plt.axis([-2, 4, -2, 4])
9 plt.xticks(fontsize=16)
10 plt.yticks(fontsize=16)
11 plt.scatter(m[0], m[1], facecolor='C0', edgecolor='k',
12             →s=100, marker="X")
13 plt.scatter(X[:,0], X[:,1], facecolor='C0', edgecolor='k',
14             →s=100)
15 plt.savefig("7.2.4.plotX.eps")
```

7.2.4.1. Suma de trozos de la matriz

- Completa el script para hallar $1_N^t X$, $X 1_D$, $1_N^t X 1_D$, $\bar{x}$ y $\bar{\bar{x}}$:

7.2.4.1.sumslices.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 N, D = X.shape
4 print(np.ones(N) @ X)
5 print(X @ np.ones(D))
6 print(np.ones(N) @ X @ np.ones(D))
7 print(np.mean(X, axis=0))
8 print(np.mean(X))
```

7.2.4.1.sumslices.out

```
1 [4. 4.]
2 [-2.  2.  2.  6.]
3 8.0
4 [1. 1.]
5 1.0
```

7.2.4.1.sumslices.alt.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 print(np.sum(X, axis=0, dtype=float))
4 print(np.sum(X, axis=1, dtype=float))
5 print(np.sum(X, dtype=float))
```

7.2.4.2. Escalado de filas y columnas de una matriz

- Completa el script para escalar X a la mitad, por filas, $\text{diag}(s)X$, y por columnas, $X \text{diag}(s)$, con una matriz de escalado s apropiada

7.2.4.2.scaling.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 N, D = X.shape
4 print((.5 * np.eye(N)) @ X)
5 print(X @ (np.eye(D) * .5))
```

7.2.4.2.scaling.out

```
1 [[-0.5 -0.5]
2  [ 0.   1. ]
3  [ 1.   0. ]
4  [ 1.5  1.5]]
5 [[-0.5 -0.5]
6  [ 0.   1. ]
7  [ 1.   0. ]
8  [ 1.5  1.5]]
```

7.2.4.3. Suma de cuadrados y matriz de dispersión

- Halla las matrices suma de cuadrados y de dispersión de $\mathbf{X}$

$$\mathbf{S}_0 = \sum_n \mathbf{x}_n \mathbf{x}_n^t = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 4 \end{bmatrix} + \begin{bmatrix} 4 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 9 & 9 \\ 9 & 9 \end{bmatrix} = \begin{bmatrix} 14 & 10 \\ 10 & 14 \end{bmatrix}$$

$$\begin{aligned} \mathbf{S}_{\bar{\mathbf{x}}} &= \sum_n (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^t \\ &= \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix} = \begin{bmatrix} 10 & 6 \\ 6 & 10 \end{bmatrix} \end{aligned}$$

► Completa el script para hallar S_0 y $S_{\bar{x}}$

7.2.4.3.sumsq+scatter.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 S0 = X.T @ X
4 print(S0)
5 N, D = X.shape
6 Sx = np.cov(X, rowvar=False, bias=True)
7 print(N*Sx)
```

7.2.4.3.sumsq+scatter.out

```
1 [[14 10]
2  [10 14]]
3 [[10.  6.]
4  [ 6. 10.]
```

7.2.4.4. Matriz de Gram

- Halla la matriz de Gram de X

$$K = XX^t = \begin{bmatrix} -1 & -1 \\ 0 & 2 \\ 2 & 0 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} -1 & 0 & 2 & 3 \\ -1 & 2 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 2 & -2 & -2 & -6 \\ -2 & 4 & 0 & 6 \\ -2 & 0 & 4 & 6 \\ -6 & 6 & 6 & 18 \end{bmatrix}$$

- Completa el script para hallar K

7.2.4.4.Gram.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 print(X.dot(X.T))
```

7.2.4.4.Gram.out

```
1 [[ 2 -2 -2 -6]
2  [-2  4  0  6]
3  [-2  0  4  6]
4  [-6  6  6 18]]
```

7.2.4.5. Matriz de distancias

- Completa el script para hallar la matriz de distancias al cuadrado entre datos en X , $D = \text{diag}(K)1_N^t - 2K + 1_N \text{diag}(K)^t$:

```
_____ 7.2.4.5.D.py _____  
1 import numpy as np  
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ], dtype=float)  
3 N, D = X.shape  
4 K = X.dot(X.T)  
5 H = np.tile(np.diag(K), (N, 1))  
6 print(H + H.T - 2*K)
```

```
_____ 7.2.4.5.D.out _____  
1 [[ 0. 10. 10. 32.]  
2  [10.  0.  8. 10.]  
3  [10.  8.  0. 10.]  
4  [32. 10. 10.  0.]
```

```
_____ 7.2.4.5.D.alt.py _____  
1 import numpy as np  
2 import scipy.spatial as spt  
3 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ], dtype=float)  
4 V = spt.distance.pdist(X, 'sqeuclidean')  
5 print(spt.distance.squareform(V))
```

7.3. Inversión de matrices

7.3.1. Inversa de una matriz cuadrada

► Halla la inversa de $\Sigma = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix}$, Σ^{-1} , y comprueba que $\Sigma\Sigma^{-1} = \mathbf{I}$

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \rightarrow \mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$$

$$\Sigma^{-1} = \frac{1}{4} \begin{bmatrix} \frac{5}{2} & -\frac{3}{2} \\ -\frac{3}{2} & \frac{5}{2} \end{bmatrix} = \begin{bmatrix} \frac{5}{8} & -\frac{3}{8} \\ -\frac{3}{8} & \frac{5}{8} \end{bmatrix}$$

$$\Sigma\Sigma^{-1} = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix} \begin{bmatrix} \frac{5}{8} & -\frac{3}{8} \\ -\frac{3}{8} & \frac{5}{8} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

7.4. Descomposición en valores propios (EVD)

7.4.1. Conceptos básicos

- Sea $A \in \mathbb{R}^{n \times n}$. Indica la afirmación *incorrecta*:
1. La suma y producto de los valores propios de A son iguales a su traza y determinante, respectivamente
 2. El rango de A es igual al número de sus valores propios no nulos
 3. Los valores propios de una matriz diagonal o triangular son los elementos de su diagonal
 4. Todas las anteriores son correctas

La 4.

- Halla los valores y vectores propios de $\Sigma = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix}$ a partir de su ecuación característica

$$|\lambda \mathbf{I} - \Sigma| = 0 \rightarrow \left| \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix} \right| = \left| \begin{matrix} \lambda - \frac{5}{2} & -\frac{3}{2} \\ -\frac{3}{2} & \lambda - \frac{5}{2} \end{matrix} \right| = 0$$

$$\rightarrow \left(\lambda - \frac{5}{2} \right)^2 - \frac{9}{4} = 0 \rightarrow \lambda^2 - 5\lambda + 4 = 0 \rightarrow \begin{cases} \lambda_1 = 4 \\ \lambda_2 = 1 \end{cases}$$

$$\Sigma \mathbf{e}_1 = \lambda_1 \mathbf{e}_1 \rightarrow \mathbf{e}_1 = \begin{pmatrix} \alpha \\ \alpha \end{pmatrix} \xrightarrow{\|\mathbf{e}_1\|=1} \mathbf{e}_1 = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix} \text{ o } \mathbf{e}_1 = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{bmatrix}$$

$$\Sigma \mathbf{e}_2 = \lambda_2 \mathbf{e}_2 \rightarrow \mathbf{e}_2 = \begin{pmatrix} -\alpha \\ \alpha \end{pmatrix} \xrightarrow{\|\mathbf{e}_2\|=1} \mathbf{e}_2 = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix} \text{ o } \mathbf{e}_2 = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \end{bmatrix}$$

7.4.3. Valores y vectores propios de matriz simétrica

► Invierte $\Sigma = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix}$ a partir de sus valores y vectores propios

$$\lambda_1 = 4 \quad \mathbf{u}_1 = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix} \quad \lambda_2 = 1 \quad \mathbf{u}_2 = \begin{bmatrix} -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}$$

$$\begin{aligned} \Sigma^{-1} &= [\mathbf{u}_1, \mathbf{u}_2] \text{diag}(1/\lambda_1, 1/\lambda_2) \begin{bmatrix} \mathbf{u}_1^t \\ \mathbf{u}_2^t \end{bmatrix} \\ &= \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} 1/4 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{8} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{8} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \\ &= \begin{bmatrix} \frac{2}{16} + \frac{2}{4} & \frac{2}{16} - \frac{2}{4} \\ \frac{2}{16} - \frac{2}{4} & \frac{2}{16} + \frac{2}{4} \end{bmatrix} = \begin{bmatrix} \frac{5}{8} & -\frac{3}{8} \\ -\frac{3}{8} & \frac{5}{8} \end{bmatrix} \end{aligned}$$

7.4.5. Estandarización y blanqueado de datos

- Estandariza $\mathbf{X}$ (sec. 7.2.4)

$$\boldsymbol{\mu} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad \boldsymbol{\Sigma} = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix} \quad \sigma_1 = \sigma_2 = \sqrt{\frac{5}{2}} = 1.5811$$

$$\begin{aligned} (\mathbf{X} - \mathbf{1}_4 \boldsymbol{\mu}^t) \text{diag}(\boldsymbol{\sigma})^{-1} &= \begin{bmatrix} -2 & -2 \\ -1 & 1 \\ 1 & -1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} \frac{1}{1.58} & 0 \\ 0 & \frac{1}{1.58} \end{bmatrix} \\ &= \begin{bmatrix} -1.26 & -1.26 \\ -0.63 & 0.63 \\ 0.63 & -0.63 \\ 1.26 & 1.26 \end{bmatrix} \end{aligned}$$

► Completa el script para estandarizar X (sec. 7.2.4)

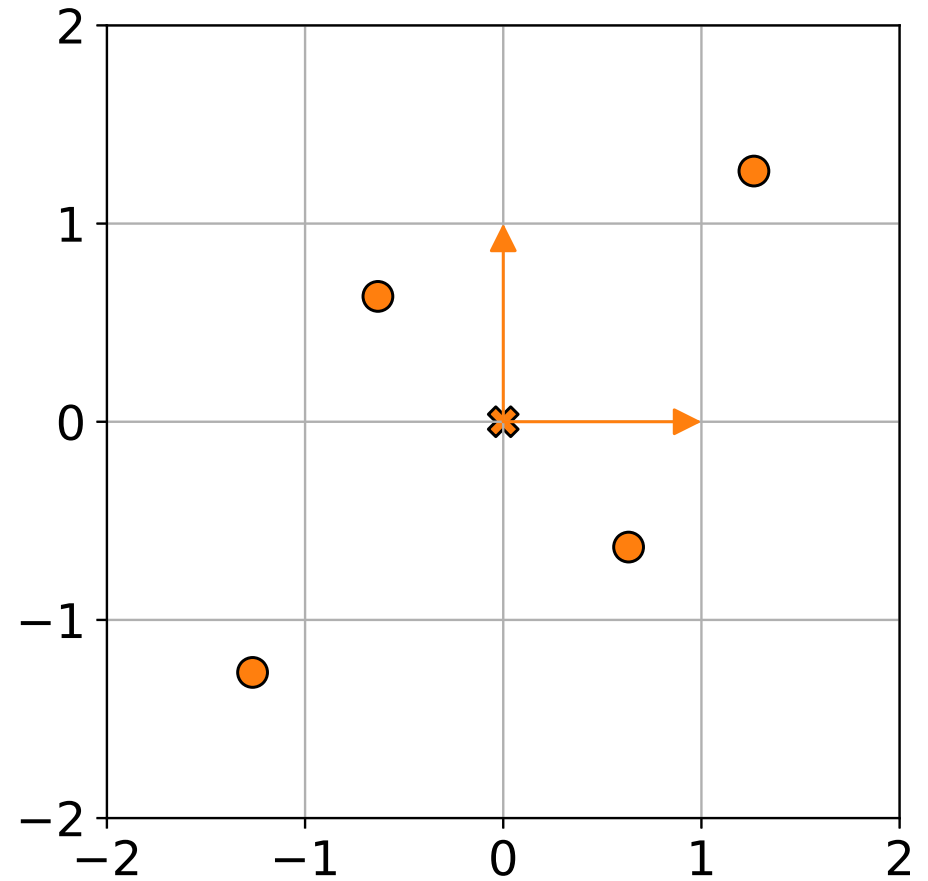
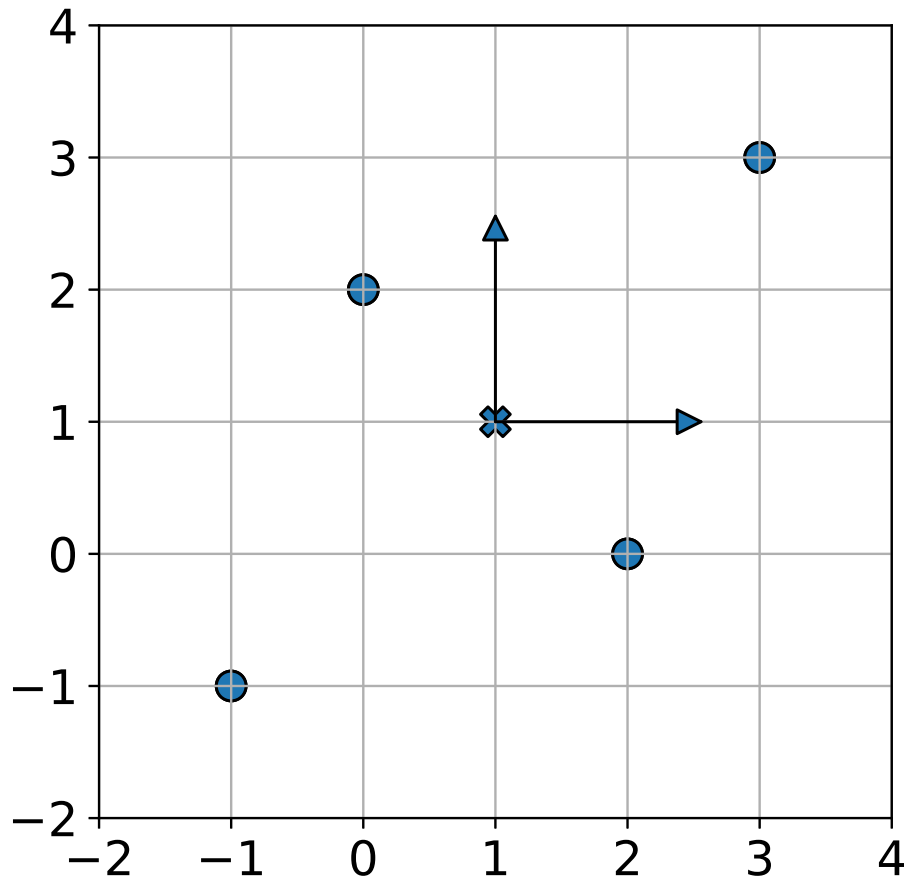
7.4.5.standardize.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 Xs = (X - np.mean(X, axis=0)) / np.std(X, axis=0)
4 print(Xs)
```

7.4.5.standardize.out

```
1 [[-1.26491106 -1.26491106]
2  [-0.63245553  0.63245553]
3  [ 0.63245553 -0.63245553]
4  [ 1.26491106  1.26491106]]
```

► Representa los datos originales y estandarizados



► PCA-blanquea $\mathbf{X}$ (sec. 7.2.4)

$$\begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix} = \mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^t = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \\ = \begin{bmatrix} 2\sqrt{2} & -\frac{\sqrt{2}}{2} \\ 2\sqrt{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} \frac{5}{2} & \frac{3}{2} \\ \frac{3}{2} & \frac{5}{2} \end{bmatrix}$$

$$\mathbf{\Sigma}^{-1} = (\mathbf{U}\mathbf{\Lambda}\mathbf{U}^t)^{-1} = \mathbf{U}\mathbf{\Lambda}^{-1}\mathbf{U}^t = (\mathbf{U}\mathbf{\Lambda}^{-\frac{1}{2}})(\mathbf{\Lambda}^{-\frac{1}{2}}\mathbf{U}^t) = \mathbf{W}_{\text{pca}}^t \mathbf{W}_{\text{pca}}$$

$$\mathbf{W}_{\text{pca}} = \mathbf{\Lambda}^{-\frac{1}{2}}\mathbf{U}^t = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{4} & \frac{\sqrt{2}}{4} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} = \begin{bmatrix} 0.35 & 0.35 \\ -0.7 & 0.7 \end{bmatrix}$$

$$\mathbf{W}_{\text{pca}}(\mathbf{X} - \mathbf{1}_4\boldsymbol{\mu}^t)^t = (\mathbf{X} - \mathbf{1}_4\boldsymbol{\mu}^t)\mathbf{W}_{\text{pca}}^t$$

$$= \begin{bmatrix} -2 & -2 \\ -1 & 1 \\ 1 & -1 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 0.35 & -0.7 \\ 0.35 & 0.7 \end{bmatrix} = \begin{bmatrix} -1.4 & 0 \\ 0 & 1.4 \\ 0 & -1.4 \\ 1.4 & 0 \end{bmatrix}$$

► Completa el script para PCA-blanquear X (sec. 7.2.4)

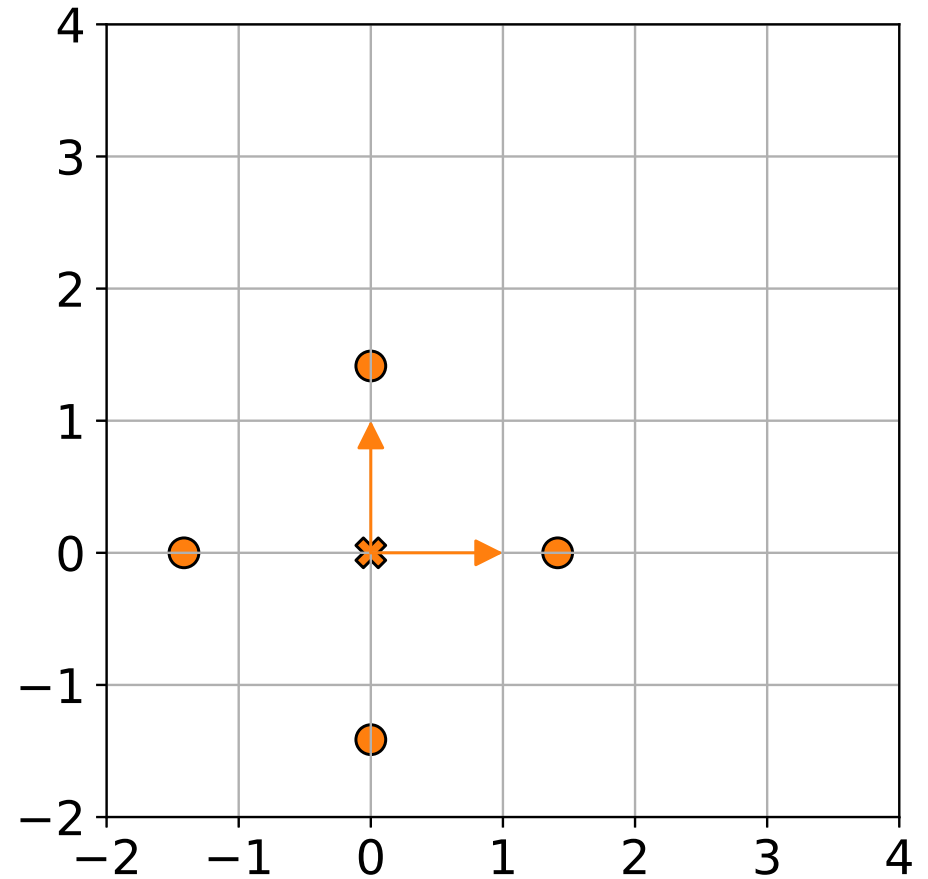
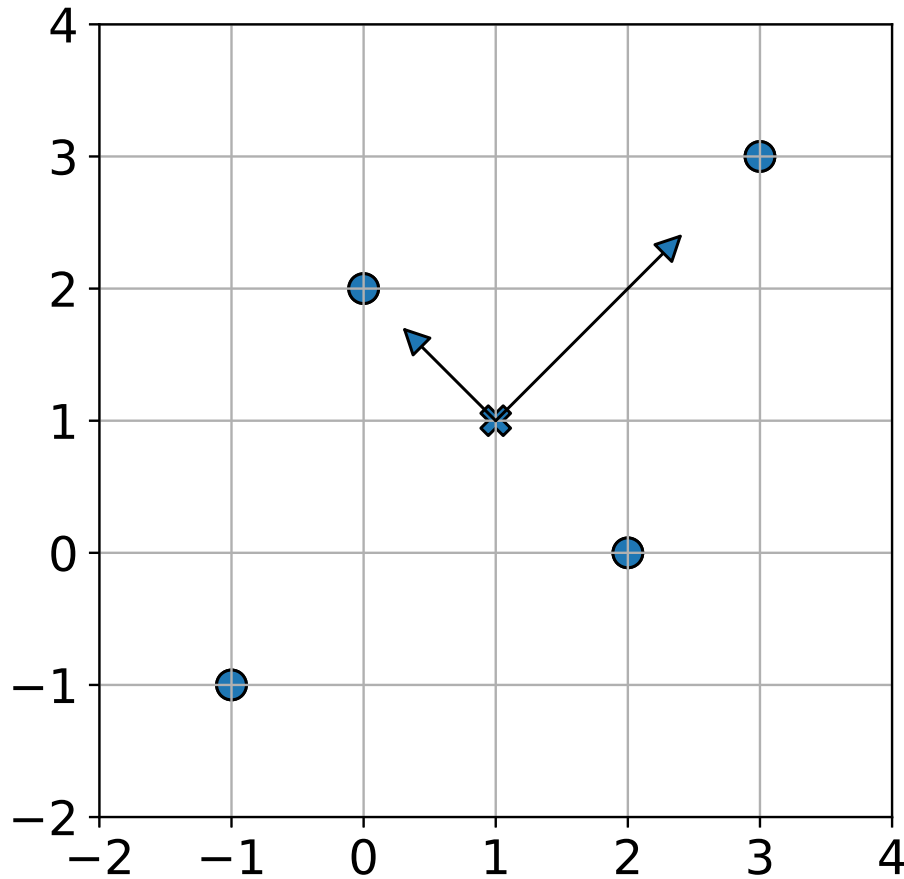
7.4.5.pca.py

```
1 import numpy as np
2 X = np.array([ [-1, -1], [0, 2], [2, 0], [3, 3] ])
3 m = np.mean(X, axis=0)
4 S = np.cov(X.T, bias=True)
5 La, U = np.linalg.eigh(S)
6 i = La.argsort()[::-1]; La = La[i]; U = U[:,i]
7 W = np.sqrt(np.linalg.inv(np.diag(La))) @ U.T
8 Xw = (X - m) @ W.T
9 print(W, Xw)
```

7.4.5.pca.out

```
1 [[ 0.35355339  0.35355339]
2  [-0.70710678  0.70710678]] [[-1.41421356  0.
3   0.          1.41421356]
4   0.          -1.41421356]
5  [ 1.41421356  0.          ]]
```


► Representa los datos originales y PCA-blanqueados



7.5. Descomposición en valores singulares (SVD)

7.5.1. Conceptos básicos

- La SVD generaliza la EVD a matrices rectangulares, $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^t = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^t + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^t$$

Indica la afirmación *incorrecta*:

1. $\mathbf{U} \in \mathbb{R}^{m \times m}$ es una matriz de columnas ortonormales ($\mathbf{U}^t \mathbf{U} = \mathbf{I}$) que llamamos vectores singulares izquierdos
2. $\mathbf{S} \in \mathbb{R}^{m \times n}$ incluye $r = \max(m, n)$ valores singulares $\sigma_i \geq 0$ en su diagonal; ceros en el resto
3. $\mathbf{V} \in \mathbb{R}^{n \times n}$ es una matriz de filas y columnas ortonormales ($\mathbf{V}^t \mathbf{V} = \mathbf{V} \mathbf{V}^t = \mathbf{I}$); sus columnas son los vectores singulares derechos
4. Son todas correctas

La 2; es $r = \min(m, n)$

7.5.2. Conexión entre la SVD y la EVD

- Completa el script para comprobar que EVD y SVD coinciden con una matriz, real, simétrica y definida positiva:

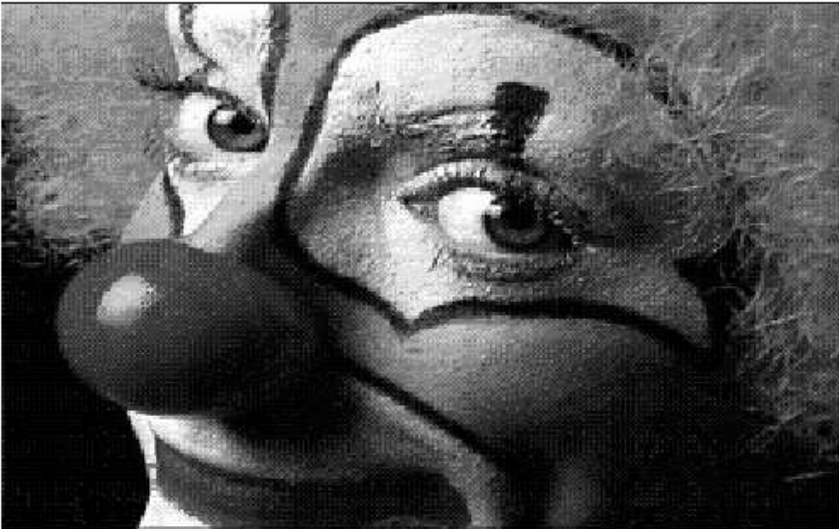
```
_____ 7.5.2.svd-evd.py _____  
1 import numpy as np  
2 A = np.array([ [5/2, 3/2], [3/2, 5/2] ])  
3 L, E = np.linalg.eigh(A)  
4 print(L, "\n", E, "\n")  
5 U, S, Vt = np.linalg.svd(A)  
6 print(U, "\n", S, "\n", Vt)
```

```
_____ 7.5.2.svd-evd.out _____  
1 [1. 4.]  
2 [[-0.70710678  0.70710678]  
3 [ 0.70710678  0.70710678]]  
4  
5 [[-0.70710678 -0.70710678]  
6 [-0.70710678  0.70710678]]  
7 [4. 1.]  
8 [[-0.70710678 -0.70710678]  
9 [-0.70710678  0.70710678]]
```

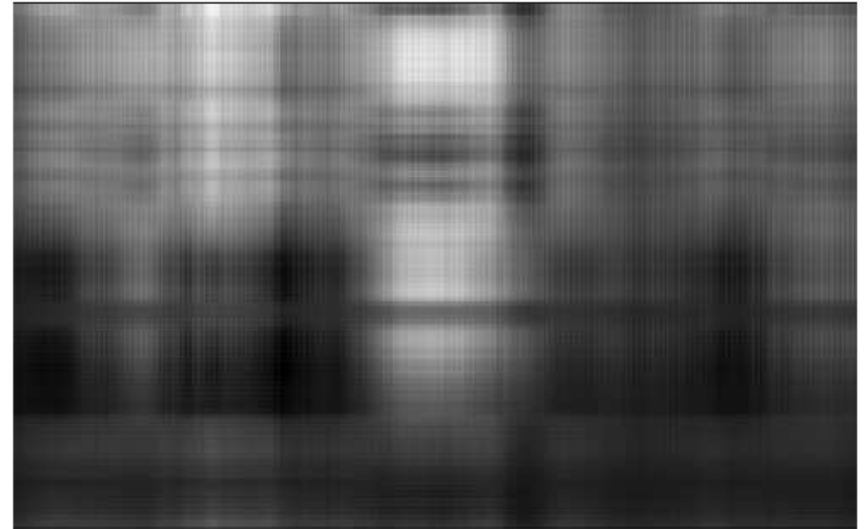
7.5.5. SVD truncada

- Reproduce el cuaderno de la [Figura 7.9](#)

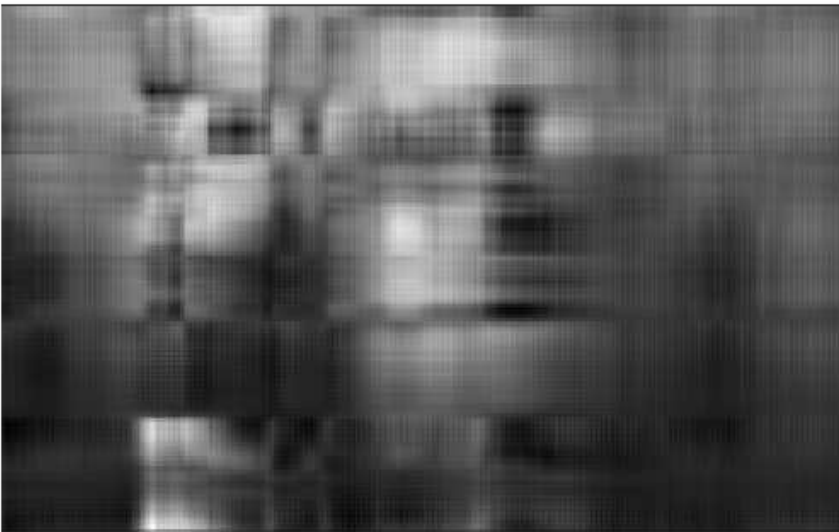
rank 200



rank 2



rank 5



rank 20

